

Hoja de trabajo #5 ? Orquestación

Curso: CC3116 • Empresa: Parachute S.A.

Solución implementada

Se construyeron tres programas que resuelven la misma solicitud: responder FAQs y, cuando el usuario solicita una cita, consultar Open-Meteo para las coordenadas 14.013722, -90.771611, validar que la fecha esté dentro de los próximos 16 días, clasificar las condiciones y calendarizar solo si el resultado es IDEAL o MARGINAL.

La calendarización se materializa mediante `schedule_tool`, que vuelve a consultar Open-Meteo como barrera de seguridad, registra la cita en `data/citas.json` con estado pendiente de confirmación del instructor y nunca confirma automáticamente una operación de salto.

La integración está centralizada en `shared/parachute.py`. Esta capa incluye y reutiliza el corpus FAQ de HDT4, valida fechas, consulta la opción daily de Open-Meteo y aplica los umbrales deterministas:

- viento superficial: ideal menor a 20 km/h, marginal de 20 a 28 km/h y prohibido sobre 28 km/h;
- ráfagas sobre 35 km/h: prohibido;
- precipitación mayor a 0 mm: prohibido;
- nubes: ideal menor a 30%, marginal de 30 a 75% y prohibido sobre 75%.

Si existe una condición prohibida o faltan datos, la recomendación es NO SEGURO / PROHIBIDO. La decisión automática no reemplaza la autoridad del instructor o piloto.

Arquitecturas

Centralizada

`centralizada.py` tiene un solo `SupervisorCentral`. El supervisor mantiene el contexto, decide si necesita consultar `WeatherWorker`, `FAQWorker` o `CalendarWorker` y llama a los workers como tools con `as_tool()`. Para citas exige clima antes de calendarizar.

Jerarquica

`jerarquica.py` usa tres niveles: `RootManager`, managers especializados (`WeatherManager`, `FAQManager`, `CalendarManager`) y workers. El manager raíz descompone la solicitud y calendarización recibe únicamente un reporte meteorológico aprobado.

Descentralizada

`descentralizada.py` no tiene manager. `IntakeAgent`, `WeatherAgent` y `FAQAgent` son pares independientes; la conversación se delega mediante handoffs. Los agentes pueden entregar una consulta al especialista correspondiente, lo que elimina el punto cénico de falla, pero requiere instrucciones y contratos muy claros para evitar ciclos o respuestas inconsistentes.

Preguntas

¿Qué arquitectura o arquitecturas resuelven mejor este problema? ¿Por qué?

La arquitectura centralizada es la mejor opción inicial para este problema. El flujo tiene pocos dominios, reglas de negocio deterministas y una secuencia sencilla: identificar intención, consultar clima si corresponde, evaluar umbrales y responder. Un único supervisor facilita trazas, auditoría y garantiza que una cita no se acepte sin revisar el clima.

La arquitectura jerárquica es la mejor candidata si Parachute S.A. seguiría agregando muchos requisitos funcionales. La separación por managers permite añadir dominios (pagos, disponibilidad, clientes, seguros o notificaciones) sin saturar un único supervisor. Su costo es mayor latencia y complejidad.

La descentralizada funciona, pero no es la más adecuada para una decisión de seguridad con reglas rígidas: los handoffs pueden ser más difíciles de auditar y controlar. Sería más atractiva para exploración, múltiples hipótesis o dominios que deban operar con mayor independencia.

¿Es necesario utilizar un sistema multiagente?

No es estrictamente necesario. Un solo agente con dos tools (FAQ y clima) o incluso un flujo determinista con una interfaz conversacional resolvería los requisitos actuales con menor costo, latencia y superficie de error.

El MAS sí es útil como ejercicio de orquestación y como preparación para el crecimiento anunciado. La especialización separa responsabilidades y permite comparar control, escalabilidad, tolerancia a fallos y trazabilidad. Aun así, la decisión meteorológica debe permanecer en código determinista compartido; los agentes deben interpretar la solicitud y coordinarse, no inventar ni modificar los umbrales.

Archivos entregados

- centralizada.py, jerarquica.py, descentralizada.py.
- shared/parachute.py con FAQ, clima, calendarización y reglas comunes.
- diagramas/centralizada.mmd, diagramas/jerarquica.mmd, diagramas/descentralizada.mmd.
- tests/test_parachute.py.

Fuente de API: documentación oficial de Open-Meteo, con referencia técnica en <https://open-meteo.com/en/docs>.